LAPORAN PRAKTIKUM

INTERNET OF THINGS (IOT) II

Praktikum 1

Disusun Oleh :

Nama : Naila Rahma Chintya Aulya

NPM : 25782020

Kelas : 3A

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, A.Md

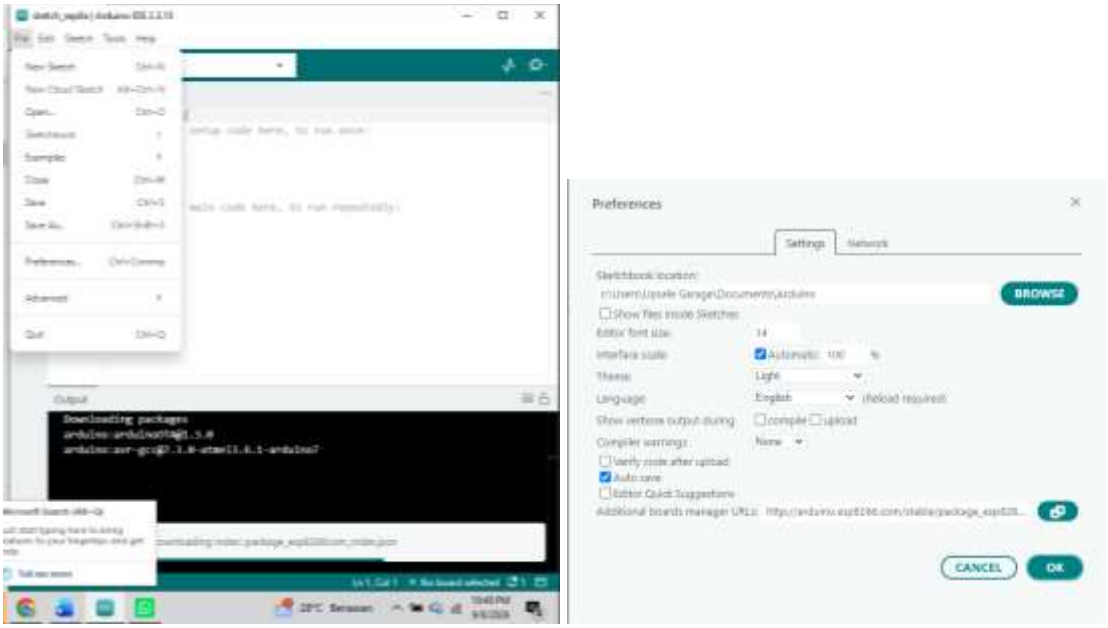
JURUSAN TEKNOLOGI INFORMASI

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

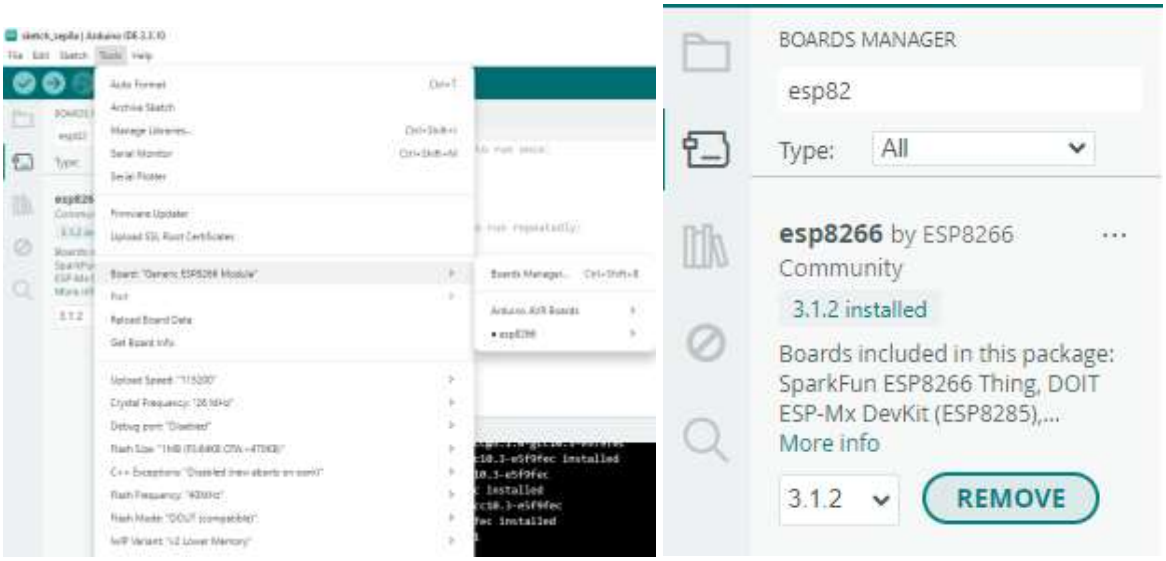
POLITEKNIK NEGERI LAMPUNG

2026/2027

Simulasi Rangkaian Praktikum



- Buka menu File lalu pilih Preferences untuk membuka jendela pengaturan utama Arduino IDE.
- Masukkan tautan modul pihak ketiga pada kolom Additional boards manager URLs, lalu klik OK untuk menyimpan.



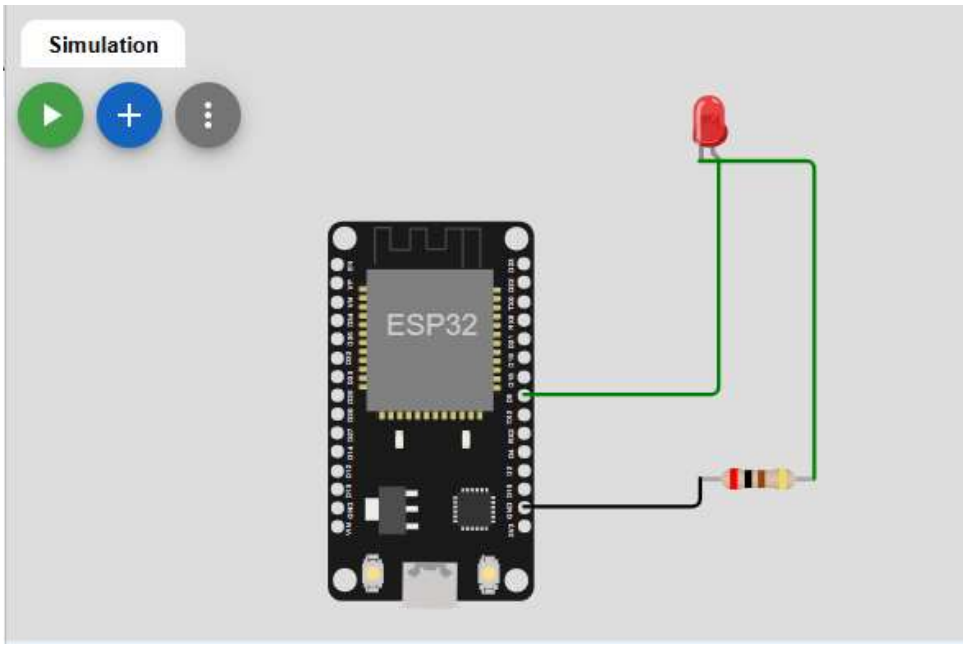
- Buka Tools > Board > Boards Manager, lalu ketik "esp8266" pada kolom pencarian untuk menemukan paket perangkat keras yang sesuai.
- Pilih dan klik Install pada paket esp8266 by ESP8266 Community agar Arduino IDE mengenali arsitektur modul tersebut.

Praktikum ini membahas dasar penggunaan mikrokontroler untuk membaca dan memberikan sinyal digital. Pada bagian praktikum, rangkaian disimulasikan dengan menghubungkan board mikrokontroler, LED, resistor, dan push button. Keterangan berikut dibuat mengikuti urutan screenshot pada file praktikum.

1.5 Praktikum

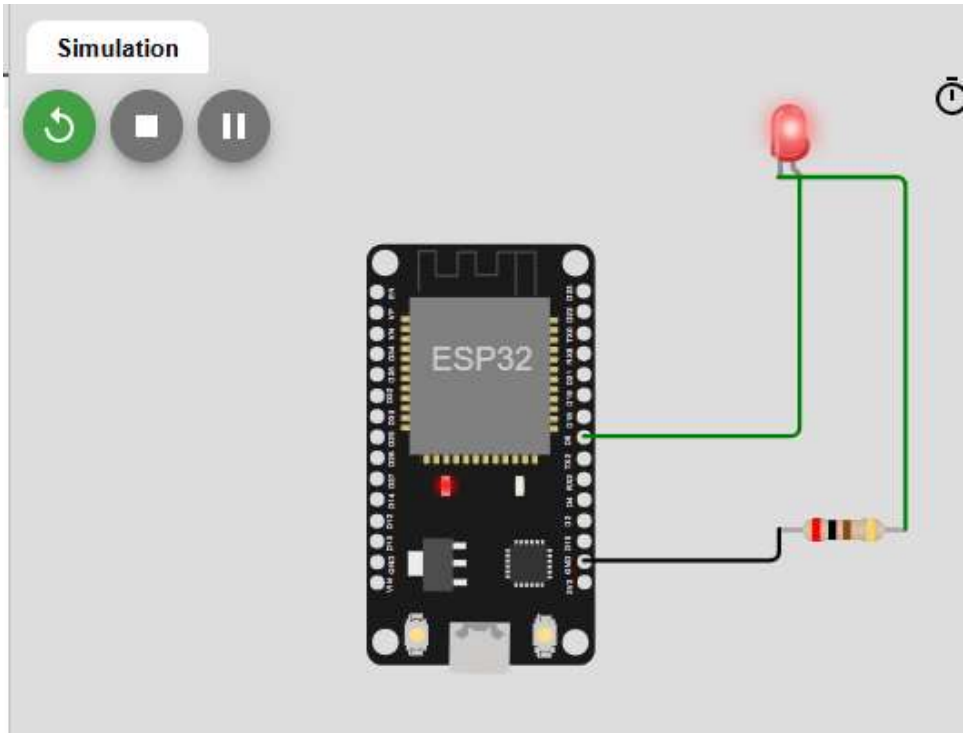
1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)

Praktikum ini bertujuan memprogram NodeMCU agar mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal.



Gambar 1. Simulasi rangkaian Digital Output

- Board mikrokontroler berfungsi sebagai otak sistem yang menjalankan program dan mengatur sinyal keluaran.
- LED digunakan sebagai indikator visual. LED menyala ketika mendapat kondisi HIGH dan mati ketika kondisi LOW.
- Resistor dipasang sebagai pembatas arus agar arus yang melewati LED tetap aman.
- Pin D1/GPIO5 menjadi jalur keluaran digital dari mikrokontroler menuju LED.
- GND menjadi jalur referensi dan penutup rangkaian listrik.



Gambar 2. Kondisi LED saat program dijalankan

Pada screenshot terlihat LED dalam keadaan menyala. Kondisi ini terjadi ketika program memberikan logika HIGH pada pin keluaran. Saat program kemudian memberikan logika LOW, LED akan mati. Dengan demikian, perubahan HIGH dan LOW pada pin dapat digunakan untuk mengontrol komponen keluaran secara sederhana.

```
Praktikum 1 - Digital Output Dimulai!
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
```

Gambar 3. Tampilan Serial Monitor Digital Output

Serial Monitor digunakan untuk melihat informasi yang dikirim oleh program. Pada praktikum ini status LED ditampilkan secara bergantian, yaitu ketika LED menyala dan ketika LED mati. Tampilan tersebut membantu memastikan bahwa program benar-benar sedang berjalan dan perubahan status terjadi sesuai perintah.

```ino
const int ledPin = 5;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  Serial.println("Praktikum 1 - Digital Output Dimulai!");
}

void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("LED Menyala");
  delay(1000);

  digitalWrite(ledPin, LOW);
  Serial.println("LED Mati");
  delay(1000);
}
```

Gambar 4. Kode program sketch.ino

File sketch.ino merupakan bagian yang berisi program untuk mengontrol LED. Program menentukan pin LED sebagai OUTPUT, kemudian memberikan HIGH untuk menyalakan LED dan LOW untuk mematikannya. Perintah delay(1000) membuat setiap kondisi berlangsung sekitar satu detik sehingga LED terlihat berkedip secara teratur.

```json
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": -70.8,
      "left": 176.6,
      "attrs": { "color": "red" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 128.75,
      "left": 192,
      "attrs": { "value": "200" }
    }
  ],
  "connections": [
    [ "esp:TX0", "$serialMonitor:RX", "", [] ],
    [ "esp:RX0", "$serialMonitor:TX", "", [] ],
    [ "esp:D5", "led1:A", "green", [ "h0" ] ],
    [ "esp:GND.1", "r1:1", "black", [ "h0" ] ],
    [ "r1:2", "led1:C", "green", [ "v0" ] ]
  ],
  "dependencies": {}
}
```

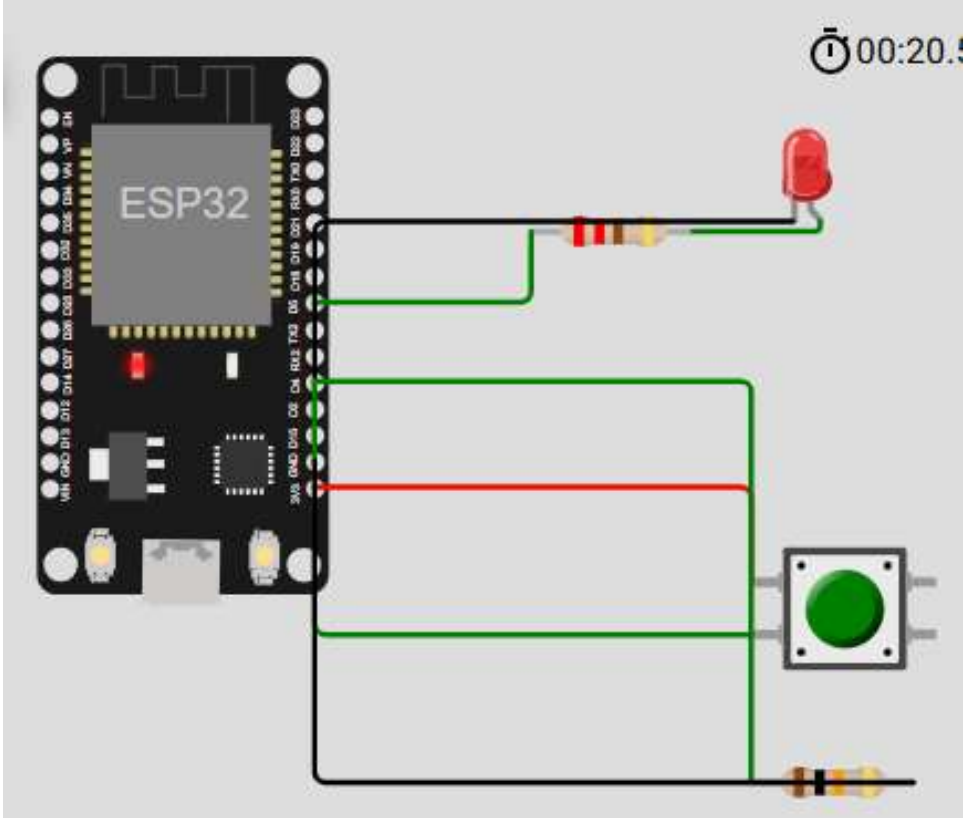
Gambar 5. Konfigurasi diagram.json

File diagram.json digunakan untuk mendefinisikan perangkat yang ada pada simulasi dan hubungan antar pin. Di dalamnya terdapat board mikrokontroler, LED, resistor, serta kabel yang menghubungkan komponen. Jadi, sketch.ino mengatur perilaku rangkaian, sedangkan diagram.json menggambarkan perangkat dan susunan rangkaiannya.

Hasil Praktikum 1: Berdasarkan simulasi, program dan rangkaian dapat bekerja bersama. LED dapat menyala dan mati mengikuti perintah digitalWrite(). Hal ini menunjukkan bahwa konsep digital output sudah dapat diterapkan pada mikrokontroler.

1.5.2 Praktikum 2: *Digital Input* (Membaca *Push Button*)

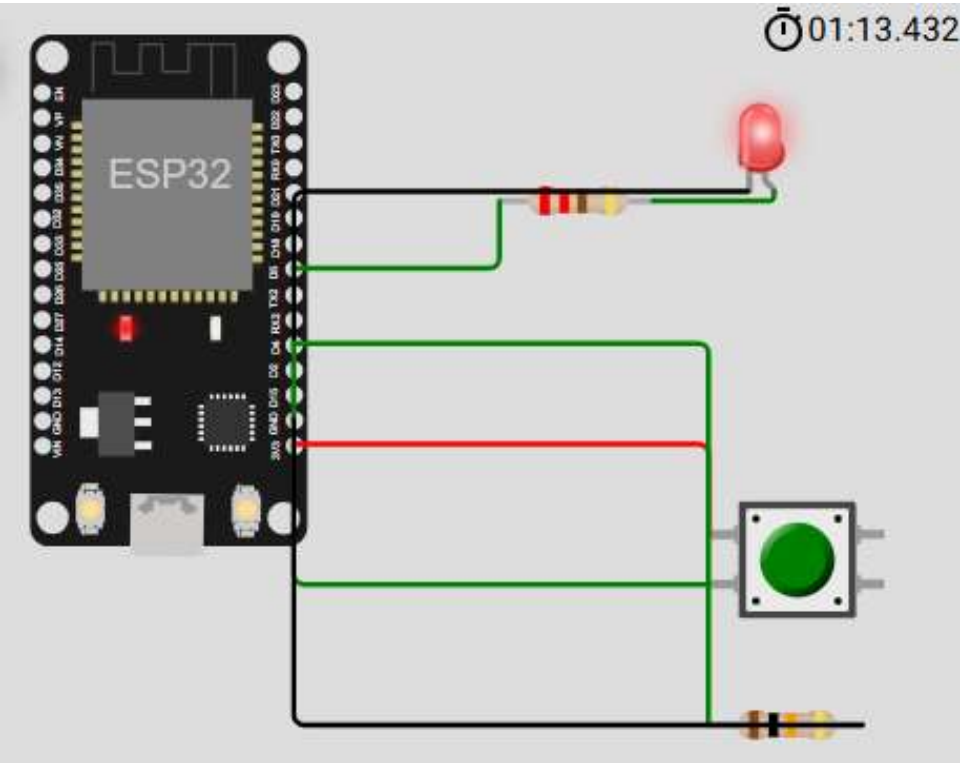
Praktikum ini bertujuan mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor *pull-down* eksternal.



Gambar 6. Simulasi rangkaian Digital Input

- Push button berfungsi sebagai input yang memberikan perubahan kondisi ketika ditekan.
- Pin D2/GPIO4 digunakan untuk membaca status tombol.
- Resistor pull-down 10kΩ menjaga input tetap LOW ketika tombol tidak ditekan.
- LED pada pin D1/GPIO5 menjadi indikator hasil pembacaan tombol.

- Saat tombol ditekan, input berubah menjadi HIGH sehingga program dapat meresponsnya.



Gambar 7. Kondisi rangkaian saat tombol ditekan

Ketika push button ditekan, tegangan dari sumber 3.3V masuk ke pin input. Program membaca kondisi tersebut sebagai HIGH. Selanjutnya LED dinyalakan sebagai tanda bahwa tombol sedang ditekan. Ketika tombol dilepas, resistor pull-down membuat input kembali LOW dan LED dimatikan.

```
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
```

Gambar 8. Tampilan Serial Monitor saat tombol ditekan

Pada Serial Monitor terlihat pesan bahwa tombol ditekan dan LED dalam keadaan ON. Pesan tersebut muncul karena program membaca kondisi input menggunakan digitalRead(). Dalam praktikum ini tombol masih bersifat momentary, sehingga LED hanya menyala selama tombol ditekan dan akan kembali mati setelah tombol dilepas.

```
1   const int buttonPin = 4;
2   const int ledPin = 5;
3
4   int buttonState = 0;
5
6   void setup() {
7     Serial.begin(115200);
8     pinMode(buttonPin, INPUT);
9     pinMode(ledPin, OUTPUT);
10  }
11
12  void loop() {
13    buttonState = digitalRead(buttonPin);
14
15    if (buttonState == HIGH) {
16      digitalWrite(ledPin, HIGH);
17      Serial.println("Tombol ditekan! -> LED ON");
18    } else {
19      digitalWrite(ledPin, LOW);
20    }
21  }
```

Gambar 9. Kode program sketch.ino Digital Input

Pada sketch.ino, pin tombol didefinisikan sebagai input dan pin LED sebagai output. Program membaca nilai tombol menggunakan digitalRead(buttonPin). Jika hasil pembacaan HIGH, LED

dinyalakan. Jika hasil pembacaan LOW, LED dimatikan. Dengan cara ini, kondisi LED mengikuti kondisi tombol secara langsung.

```
{
  "version": 1,
  "author": "Praktikum IoT",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-esp32-devkit-v1",
      "id": "esp",
      "top": -80,
      "left": -80,
      "attrs": {}
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": -60,
      "left": 180,
      "attrs": {
        "color": "red"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -20,
      "left": 100,
      "attrs": {
        "value": "220"
      }
    },
```

Gambar 10. Bagian awal konfigurasi diagram.json

Diagram rangkaian memuat board mikrokontroler, LED, resistor, push button, dan kabel penghubung. Komponen-komponen tersebut disusun sesuai dengan konsep input pull-down yang digunakan pada praktikum.

```
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 100,
      "left": 180,
      "attrs": {
        "color": "green"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": 180,
      "left": 180,
      "attrs": {
        "value": "10000"
      }
    }
  ],
  "connections": [
    [
      "esp:D5",
      "r1:1",
      "green",
      []
    ],
    [
      "r1:2",
      "led1:A",
      "green",
      []
    ],
    [
      "led1:C",
      "esp:GND.1",
      "black",
      []
    ],
    [
      "esp:3V3",
      "btn1:1.l",
      "red",
      []
    ],
    [
      "btn1:2.l",
      "esp:D4",
      "green",
      []
    ],
    [
      "esp:D4",
      "r2:1",
      "green",
      []
    ],
```

```
    [
      "r2:2",
      "esp:GND.1",
      "black",
      []
    ],
    [
      "esp:TX0",
      "$serialMonitor:RX",
      "",
      []
    ],
    [
      "esp:RX0",
      "$serialMonitor:TX",
      "",
      []
    ]
  ],
  "dependencies": {}
}
```

Gambar 11. Lanjutan konfigurasi diagram.json

Bagian lanjutan diagram.json menunjukkan koneksi antara pin mikrokontroler dengan komponen input dan output. Kesesuaian nomor pin pada program dengan jalur kabel pada simulasi penting agar tombol dan LED dapat bekerja sesuai yang diharapkan.

Hasil Praktikum 2: Push button berhasil digunakan sebagai input digital. Saat tombol ditekan, mikrokontroler membaca HIGH dan LED menyala. Saat tombol dilepas, input kembali LOW dan LED mati. Rangkaian pull-down membuat kondisi input tetap jelas ketika tombol tidak ditekan.

1.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda. Jelaskan hasilnya di dalam laporan!

1. Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Jawab : Jika menggunakan rangkaian Active-High atau pull-down dan tombol dihubungkan ke pin D8 (GPIO 15), lalu tombol ditekan tepat saat NodeMCU baru diberi daya, NodeMCU dapat mengalami masalah saat proses booting. GPIO 15 merupakan salah satu pin yang berperan dalam menentukan kondisi awal saat NodeMCU melakukan boot. Jika kondisi logikanya tidak sesuai, NodeMCU dapat gagal melakukan proses booting atau tidak menjalankan program dengan normal. Oleh karena itu, GPIO 15 sebaiknya tidak digunakan sembarangan untuk tombol yang dapat mengubah kondisi logika saat NodeMCU dinyalakan.

2. Latihan 2 (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi delay() yang harus Anda ubah.

Jawab : Agar LED menyala selama 200 ms dan mati selama 800 ms, dua baris fungsi delay() diubah menjadi:

```
delay(200);

delay(800);
```

Baris pertama digunakan untuk menentukan lama LED menyala, sedangkan baris kedua menentukan lama LED mati.

3. Latihan 3 (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Jawab : Jika kaki LED dipasang terbalik, yaitu Anoda dihubungkan ke GND dan Katoda ke D1, arus listrik tidak dapat mengalir melalui LED dalam arah yang benar. Akibatnya, LED tidak menyala.

Program tidak mengalami error saat kompilasi karena susunan kaki LED merupakan masalah pada rangkaian fisik, bukan kesalahan pada kode program. LED tidak menyala karena LED hanya dapat bekerja jika arus mengalir dari Anoda menuju Katoda dengan arah yang sesuai.

4. Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

Jawab : Perilaku LED dapat dibalik dengan mengubah kondisi if-else. Jika tombol menggunakan logika Active-High, contohnya:

```
if (digitalRead(buttonPin) == LOW) {
  digitalWrite(ledPin, HIGH);
} else {
  digitalWrite(ledPin, LOW);
}
```

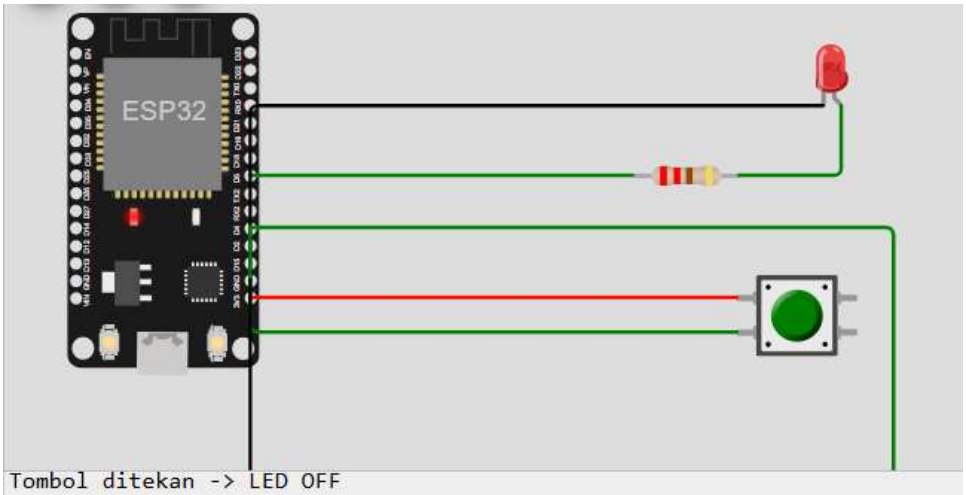
Dengan kondisi tersebut, ketika tombol **tidak ditekan** dan input bernilai LOW, LED akan **menyala**. Ketika tombol **ditekan** dan input berubah menjadi HIGH, LED akan **mati**.

1.7 Tugas

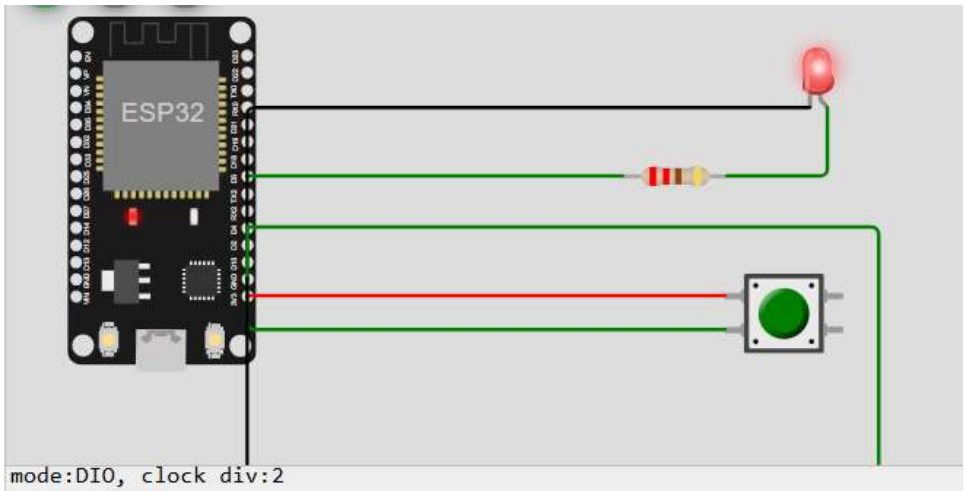
Tugas ini harus dikerjakan secara mandiri untuk mengukur kemampuan integrasi dan logika pemrograman Anda.

Modifikasi Sistem Sakelar *Toggle* (*Latching*)

- **Tujuan:** Mampu mengimplementasikan logika pelacakan status (*state tracking*) dan membedakan antara perilaku tombol *momentary* dengan *latching*.
- **Tingkat Kesulitan:** Menengah
- **Instruksi:**
    i. Gunakan konfigurasi *hardware* yang persis sama dengan Praktikum 2 (Rangkaian *Pull-Down*).
    ii. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
        - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
        - Jika tombol ditekan 1 kali *lalu dilepas*, LED **menyala dan tetap menyala**.
        - Jika tombol ditekan 1 kali lagi *lalu dilepas*, LED **mati dan tetap mati**.
    iii. *Petunjuk:* Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi delay() singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada tombol fisik).
- **Kriteria Keberhasilan:**
    - Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan *satu kali tekanan singkat* (bukan ditahan).
    - Tidak ada fenomena *flickering* (lampu berkedip acak atau gagal mati) saat tombol ditekan.
    - Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan *state* dan *debounce* bekerja.




```
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
```




```
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Tombol ditekan -> LED ON
```

```json
{
  "version": 1,
  "author": "Praktikum IoT",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-esp32-devkit-v1",
      "id": "esp",
      "top": -81.7,
      "left": -283.4,
      "attrs": {}
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": -70.8,
      "left": 119,
      "attrs": {
        "color": "red"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 3.95,
      "left": 28.8,
      "attrs": {
        "value": "220"
      }
    },
```

```
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 63.8,
      "left": 86.4,
      "attrs": {
        "color": "green"
      }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": 176.75,
      "left": 172.8,
      "attrs": {
        "value": "10000"
      }
    }
  ],
  "connections": [
    [
      "esp:D5",
      "r1:1",
      "green",
      []
    ],
    [
      "r1:2",
```

```
      "led1:A",
      "green",
      []
    ],
    [
      "led1:C",
      "esp:GND.1",
      "black",
      []
    ],
    [
      "esp:3V3",
      "btn1:1.l",
      "red",
      []
    ],
    [
      "btn1:2.l",
      "esp:D4",
      "green",
      []
    ],
    [
      "esp:D4",
      "r2:1",
      "green",
      []
    ],
```

```
    [
      "r2:2",
      "esp:GND.1",
      "black",
      []
    ],
    [
      "esp:TX0",
      "$serialMonitor:RX",
      "",
      []
    ],
    [
      "esp:RX0",
      "$serialMonitor:TX",
      "",
      []
    ]
  ],
  "dependencies": {}
}
```

```ino
const int buttonPin = 4;
const int ledPin = 5;

// Menyimpan status LED, awalnya OFF
bool ledState = false;

// Menyimpan kondisi tombol sebelumnya
int lastButtonState = LOW;

void setup() {
  Serial.begin(115200);

  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);

  // Saat sistem pertama dinyalakan, LED mati
  digitalWrite(ledPin, LOW);
}

void loop() {
  int buttonState = digitalRead(buttonPin);

  // Mendeteksi tombol baru ditekan
  if (buttonState == HIGH && lastButtonState == LOW) {

    // Membalik status LED: OFF menjadi ON, ON menjadi OFF
    ledState = !ledState;

    digitalWrite(ledPin, ledState);

    if (ledState == true) {
      Serial.println("Tombol ditekan -> LED ON");
    } else {
      Serial.println("Tombol ditekan -> LED OFF");
    }

    // Debounce agar satu tekanan tidak terbaca berkali-kali
    delay(200);
  }

  // Menyimpan kondisi tombol sebelumnya
  lastButtonState = buttonState;
}
```